Designing Deep Learning Solutions for Real-World Deployment

Rowan Alhallak

Indiana Tech

I. Introduction and Problem Statement

My goal for this project was to develop a multiclass image classification model that can accurately identify distinct cat and dog breeds, but specifically be able to do so in a mobile environment using smartphone photography. My initial inspiration for this project came from the observations I made during my CS 5700 Module 4 assignment. For that assignment I used a ResNet-18 architecture and the CIFAR-10 dataset. Though the model has fairly high overall accuracy, I noticed that it struggled specifically with differentiating between cats and dogs. This inspired me to look deeper into the problem of cat and dog identification to see what possible solutions there were to the difficulty of identifying differences between otherwise very similar animals.

I specifically chose to focus this project on creating a model that could be deployed on a mobile edge device as that realistically seemed like the most relevant application of such a deep learning system. I envisioned an app where a user could either upload a photo on their phone or capture an image directly with their device's camera and be able to identify what cat or dog breed they were looking at. Unlike a deep learning system connected to the cloud, I chose a system that could process the images locally on a user's device. Not only does this solution offer data privacy to the user, but it allows offline functionality, which is a definite benefit. Because of these specifics I selected for this project, my chosen model absolutely must have a strict balance between having a small memory footprint, low computational overhead, and it must still have the ability to spot differences in fine-grained biological traits between visually similar dog and cat breeds.

II. Dataset Description

As I was specifically curious about the identification of dog and cat breeds, instead of reusing the CIFAR-10 dataset, I switched to using the Oxford-IIIT Pet Dataset, which had originally been introduced by Parkhi et al. [3]. This dataset had 37 distinct breeds of cats and dogs with about 200 images per class. This fine-grained dataset was originally used for image classification using traditional computer vision methods featuring manually developed feature extraction pipelines, not deep convolutional neural networks. One significant disadvantage of this dataset is its rather small size compared to the CIFAR-10 dataset. To avoid overfitting, I used an augmentation pipeline during training, which used random cropping and horizontal flipping in an attempt to artificially increase the dataset's visual diversity.

III. Neural Architecture Selection and Justification

As this project has a secondary goal of being small and efficient enough to deploy on an offline device, it was essential to carefully select an efficient architecture. Though convolutional neural networks like the ResNet18 I used on my CIFAR-10 project are excellent at feature-extraction, they would not be compatible with my project due to their huge parameter counts (around 11.7 million) and high battery use. My first choice in architecture was therefore the super lightweight MobileNetV2. MobileNetV2 is specifically designed for mobile edge computing and is able to keep parameter counts down to about 2.2 million.

A. Version 1 — My initial iteration for this experiment used MobileNetV2 with completely frozen base layers. Benchmarking confirmed the promised efficiency of the model, with exactly 2,271,269 parameters and a file size of just 8.57 MB. During a simulated smartphone CPU testing, the model had an average inference latency of 43.81 ms per image and a throughput of 22.83 frames per second. However, with only the classification head trained for 20 epochs using a StepLR schedule, this model configuration reached a plateau very quickly and stayed at 86.2% validation accuracy. Despite the model's speed and efficiency, it appeared to lack the ability to capture the fine-grained cat and dog textures without significant adjustments.

B. Version 2 — In order to determine whether the results I saw with my initial iteration were largely

due to the architecture or to the training methodology, I created a second iteration where I changed the training strategy used, keeping the MobileNetV2 model, but using a two-phase transfer learning strategy.

1) Phase 1: For this first stage of training, the base convolution layers were left frozen and a new linear classification head was trained for 5 epochs only. As Yosinski et al. [5] showed, the early and middle layers of a CNN serve more as general feature extractors and I followed their example by freezing those early layers to keep any large gradients from the new classification head from wrecking havoc on the pre-trained weights.

2) Phase 2: At this point, I unfroze the entire network so that I could begin fine tuning the model. At a reduced learning rate (1×10^{-4}) I trained for 10 more epochs. This method was again demonstrated by Yosinski et al. [5], where they showed that generalization improves when transferred features are fine-tuned like this. I also used a ReduceLROnPlauteau scheduler which reduced the learning rate any time the generalization stalled.

C. Version 3 — I did see some improvement with version 2, but not as much as I had hoped, so for version 3 I chose to upgrade the architecture used for my model to EfficientNet-B0, guided by the research of Tan and Le [4]. While Tan and Le [4] were primarily focused on finding an alternative architecture to ResNet (such as I used in my Module 4 assignment) and critiqued the method of scaling CNNs by depth alone, their solution, EfficientNet-B0, was sufficiently efficient to meet my goals. EfficientNet-B0 was designed to scale uniformly across depth, width, and input resolution while maintaining a degree of computational efficiency. Though it does have more parameters than MobileNetV2 (about 4 million rather than 2.2), it is still significantly smaller than ResNet-18 and in the end it gave my model the needed boost in accuracy. Because I already saw improvement with the two phase training, I continued the same routine for my version 3 model.

IV. Module Evaluation and Trade-Off Analysis

The baseline MobileNetV2 model (version 1) stalled at 86.2% validation accuracy and though it never overfit, it never increased accuracy either. After I changed my model to use the two-phase training pipeline (Version 2), the MobileNetV2 model did indeed see an improvement with a final validation accuracy of 89.5%. My personal hope was to see a final validation accuracy of at least 90% and I saw that with my Version 3 model that combined the successful two-phase training with the EfficientNet-B0 architecture. This version finally climbed past 90% and achieved a peak validation accuracy of 91.91%.

Though the accuracy was what I was looking for with Version 3, I still needed to determine if it was suitable for a mobile edge environment. In order to do this I first had to package my code in a format and language that could run on a mobile phone. I converted the model's architecture into TorchScript using serialization, decoupling the neural network from the Python interpreter and translating it into a model able to run on either iOS or Android. Once this was done I was able to capture speed and efficiency metrics on the TorchScript version of my EfficientNet-B0 model as follows:

- Mobile Model File Size: 15.37 MB
- Total Parameters: 4,054,945
- Average CPU Inference Latency: 43.54 ms per image
- Throughput: 22.97 Frames Per Second (FPS)

Despite the higher parameter count and file size compared to the MobileNetV2 model, the difference between the CPU inference Latency and the Throughput were negligible. And importantly, at 23 FPS, the EfficientNet-B0 model is fast enough to process a smartphone camera feed without noticeable lag.

Though the overall success of this model in identifying cat and dog breeds with higher validation accuracy than my previous model did, there still exists specific performance gaps worth noting. For many specific breeds, the model achieved between 98% and 100% accuracy (e.g. Leonberger and Pug). However the model's failure occurred when breeds were mainly distinguishable by size, rather than other physical features. When attempting to differentiate between the American Pit Bull Terrier and the Staffordshire Bull Terrier, the model frequently misclassified the American for the Staffordshire. As these two dog breeds have identical fur texture and snout shape, the model is left with nothing much to go on aside from how large the breeds are. Unfortunately, the model does not account necessarily for environmental size differences, limiting its ability for accuracy in this way.

V. Deployment Solution

My proposed deployment strategy is to build an on-device environment for my model using PyTorch

Mobile. As mobile environments do not come with native Python interpreters, I serialized the trained EfficientNet-B0 model into TorchScript. This TorchScript can run on its own without needing Python. Before the TorchScript can be put on a phone, though, it must also be optimized for a low energy mobile environment so that it does not drain the battery or take up too much storage. The optimize_for_mobile function was used in order to combine the multiple mathematical operations into fewer, more efficient steps. Chang et al. [1] showed that performing this “layer fusion” is highly effective in reducing the memory and increasing the speed of the model. The resulting file is a 15.37 MB optimized model which can be placed directly into a mobile phone’s application code folder. It should then be connected to an app that when a user takes a photo of a cat or dog, will automatically resize the photo, pass it to the file where the model will predict the breed and display the results on the screen without needing to connect to the internet or a cloud server.

VI. Possible Challenges and Limitations

The main limitation I uncovered so far is the difficulty the model has with biologically similar breeds. Unless there are specific context cues, the application will most likely classify terriers and specific cat breeds.

VII. Recommendations for Future Improvements

Though the EfficientNet-B0 model gave the superior validation accuracy, the size of the over 4 million parameters is still 15.37 MB and does need a fair amount of computing power to multiply together. Each of those 4 million parameters are made up of 32-bit floats, but it could be possible to reduce the size by using quantization, which would round the 32-bit floats down to 8-bit integers. There is a possible drawback to this plan, though. While it has the potential to reduce the file size by as much as 75% and maybe double the FPS, by rounding the math I would definitely lose a small amount of precision. For many environments I might not want to risk precision, but if the accuracy loss was small enough (less than 1%), then being able to reduce an app size by 75% might actually be very worth it.

References

[1] Y. Chang, J. H. Kim, H. W. Shin, C. Ha, S. Y. Lee, and T. Go, "Diagnosis of pressure ulcer stage using on-device AI," *Applied Sciences*, vol. 14, no. 16, p. 7124, 2024. doi: 10.3390/app14167124.

[2] N. P. Lopes, "Torchy: A tracing JIT compiler for PyTorch," in *Proc. 32nd ACM SIGPLAN Int. Conf. Compiler Construct.*, 2023. doi: 10.1145/3578360.3580266.

[3] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, "Cats and dogs," in *IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2012, pp. 3498-3505.

[4] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6105-6114.

[5] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 27, 2014, pp. 3320-3328.